ANEXOS TECNICOS

Sistema Digital de Atenciones e Indicadores SCI

Oficina General de Bienestar Universitario (OGBU) - UNMSM

13 de agosto de 2026

Servicios comprendidos: Alimentacion (Comedor), Servicio Social y Residencias Universitarias

ESTADO Y ALCANCE DEL SISTEMA - PERIODO REPORTADO: JULIO 2026

Este documento presenta un PROTOTIPO FUNCIONAL, desarrollado y sometido a pruebas en agosto de 2026. Al cierre de julio de 2026 el sistema no se encontraba implementado ni en uso operativo por las oficinas y unidades de la OGBU; por lo tanto, no se adjunta evidencia de uso correspondiente a ese periodo porque el aplicativo aun no habia sido desplegado.

Los datos mostrados en los Anexos 2 y 3 (capturas de pantalla, registros e indicadores) son datos de prueba (dummy), generados para validar el funcionamiento del sistema durante su desarrollo. No corresponden a atenciones reales de estudiantes.

Anexo 1	Script y Modelo Fisico de Base de Datos
Anexo 2	Capturas de Pantalla del Formulario de Registro de Atenciones
Anexo 3	Capturas de Pantalla del Dashboard de Indicadores SCI
Anexo 4	Especificacion de APIs Backend

Anexo 1 - Script y Modelo Fisico de Base de Datos

Estructura relacional que soporta el Registro Digital de Atenciones y las Encuestas de Satisfaccion. El modelo esta compuesto por 7 tablas con claves primarias autoincrementales, claves foraneas con reglas de integridad referencial (RESTRICT/CASCADE/SET NULL), indices sobre las columnas de filtrado frecuente (fecha_atencion, servicio_id) y restricciones CHECK para los valores enumerados (estado_atencion, tipo_respuesta).

1.1 Resumen del modelo (entidades y llaves)

Tabla	Clave primaria	Descripcion	Claves foraneas (FK)
usuarios_responsables	usuario_id	Operadores/trabajadores que registran atenciones	-
estudiantes	estudiante_id	Padron de estudiantes atendidos por la OGBU	-
servicios_ogbu	servicio_id	Catalogo de servicios (Alimentacion, Servicio Social, Residencias)	-
trimestres	trimestre_id	Periodos trimestrales usados para agrupar encuestas	-
atenciones	atencion_id	Registro de cada atencion brindada	estudiante_id, servicio_id, usuario_responsable_id
preguntas_encuesta	pregunta_id	Preguntas de satisfaccion por servicio y trimestre	servicio_id, trimestre_id
respuestas_encuesta	respuesta_id	Respuestas de satisfaccion (Likert 1-5 o texto libre)	pregunta_id, atencion_id (nullable)

1.2 Relaciones (claves foraneas)

```
atenciones.estudiante_id -> estudiantes.estudiante_id  (ON DELETE RESTRICT)
atenciones.servicio_id -> servicios_ogbu.servicio_id  (ON DELETE RESTRICT)
atenciones.usuario_responsable_id -> usuarios_responsables.usuario_id  (ON DELETE RESTRICT)
preguntas_encuesta.servicio_id -> servicios_ogbu.servicio_id  (ON DELETE CASCADE)
preguntas_encuesta.trimestre_id -> trimestres.trimestre_id  (ON DELETE CASCADE)
respuestas_encuesta.pregunta_id -> preguntas_encuesta.pregunta_id  (ON DELETE CASCADE)
respuestas_encuesta.atencion_id -> atenciones.atencion_id  (ON DELETE SET NULL, nullable)
```

1.3 Script DDL (MySQL 8.0 - motor desplegado y probado)

El siguiente script crea la totalidad de tablas, llaves e indices. Version equivalente para PostgreSQL disponible en el repositorio del proyecto (database/schema_postgresql.sql). Los INSERT de datos de prueba (dummy) se encuentran en el mismo archivo fuente y se omiten aqui por brevedad.

```
-- ================================================================================
-- Sistema Digital de Atenciones e Indicadores SCI - OGBU UNMSM
-- Motor: MySQL 8.0+ / MariaDB 10.5+
-- Contiene: DDL (tablas, PK, FK, indices, constraints) + datos DUMMY de prueba
-- Diferencias clave frente a la version PostgreSQL:
--   * AUTO_INCREMENT en lugar de SERIAL
--   * ENGINE=InnoDB + utf8mb4 explicitos
--   * CHECK constraints soportados desde MySQL 8.0.16 (se mantienen igual)
-- ================================================================================
```

```
SET NAMES utf8mb4;
SET FOREIGN_KEY_CHECKS = 0;

-- ------------------------------------------------------------------------
-- 1. USUARIOS_RESPONSABLES
-- ------------------------------------------------------------------------
CREATE TABLE usuarios_responsables (
    usuario_id           INT AUTO_INCREMENT PRIMARY KEY,
    nombres              VARCHAR(100) NOT NULL,
    apellidos            VARCHAR(100) NOT NULL,
    correo_institucional VARCHAR(150) NOT NULL UNIQUE,
    rol                  VARCHAR(50)  NOT NULL DEFAULT 'Operador',
    estado               TINYINT(1)   NOT NULL DEFAULT 1,
    creado_en            TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- ------------------------------------------------------------------------
-- 2. ESTUDIANTES
-- ------------------------------------------------------------------------
CREATE TABLE estudiantes (
    estudiante_id         INT AUTO_INCREMENT PRIMARY KEY,
    codigo_estudiante     VARCHAR(20)  NOT NULL UNIQUE,
    nombres               VARCHAR(100) NOT NULL,
    apellidos             VARCHAR(100) NOT NULL,
    facultad              VARCHAR(150) NOT NULL,
    correo_institucional  VARCHAR(150) NOT NULL UNIQUE,
    estado                TINYINT(1)   NOT NULL DEFAULT 1,
    creado_en             TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_estudiante_correo CHECK (correo_institucional LIKE '%@%'),
    INDEX idx_estudiantes_facultad (facultad)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- ------------------------------------------------------------------------
-- 3. SERVICIOS_OGBU
-- ------------------------------------------------------------------------
CREATE TABLE servicios_ogbu (
    servicio_id     INT AUTO_INCREMENT PRIMARY KEY,
    nombre_servicio VARCHAR(100) NOT NULL UNIQUE,
    descripcion     VARCHAR(255),
    estado          ENUM('Activo', 'Inactivo') NOT NULL DEFAULT 'Activo'
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- ------------------------------------------------------------------------
-- 4. TRIMESTRES
-- ------------------------------------------------------------------------
CREATE TABLE trimestres (
    trimestre_id      INT AUTO_INCREMENT PRIMARY KEY,
    anio              SMALLINT NOT NULL,
    periodo_trimestre TINYINT NOT NULL,
    fecha_inicio      DATE,
    fecha_fin         DATE,
    CONSTRAINT chk_periodo_trimestre CHECK (periodo_trimestre BETWEEN 1 AND 4),
    CONSTRAINT uq_trimestre_anio_periodo UNIQUE (anio, periodo_trimestre)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- ------------------------------------------------------------------------
-- 5. ATENCIONES
-- ------------------------------------------------------------------------
CREATE TABLE atenciones (
    atencion_id            INT AUTO_INCREMENT PRIMARY KEY,
    estudiante_id          INT NOT NULL,
    servicio_id            INT NOT NULL,
    fecha_atencion         TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    descripcion_motivo     TEXT NOT NULL,
    diagnostico_accion     TEXT,
    usuario_responsable_id INT NOT NULL,
    estado_atencion        ENUM('En Proceso', 'Completado') NOT NULL DEFAULT 'En Proceso',
    creado_en              TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_atenciones_estudiante FOREIGN KEY (estudiante_id)
        REFERENCES estudiantes (estudiante_id) ON DELETE RESTRICT,
    CONSTRAINT fk_atenciones_servicio FOREIGN KEY (servicio_id)
        REFERENCES servicios_ogbu (servicio_id) ON DELETE RESTRICT,
    CONSTRAINT fk_atenciones_usuario FOREIGN KEY (usuario_responsable_id)
        REFERENCES usuarios_responsables (usuario_id) ON DELETE RESTRICT,
```

```
    INDEX idx_atenciones_estudiante (estudiante_id),
    INDEX idx_atenciones_servicio (servicio_id),
    INDEX idx_atenciones_fecha (fecha_atencion),
    INDEX idx_atenciones_servicio_fecha (servicio_id, fecha_atencion)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- ------------------------------------------------------------------------
-- 6. PREGUNTAS_ENCUESTA
-- ------------------------------------------------------------------------
CREATE TABLE preguntas_encuesta (
    pregunta_id     INT AUTO_INCREMENT PRIMARY KEY,
    servicio_id     INT NOT NULL,
    trimestre_id    INT NOT NULL,
    texto_pregunta  VARCHAR(500) NOT NULL,
    tipo_respuesta  ENUM('Likert_1_5', 'Texto') NOT NULL,
    activa          TINYINT(1) NOT NULL DEFAULT 1,
    CONSTRAINT fk_preguntas_servicio FOREIGN KEY (servicio_id)
        REFERENCES servicios_ogbu (servicio_id) ON DELETE CASCADE,
    CONSTRAINT fk_preguntas_trimestre FOREIGN KEY (trimestre_id)
        REFERENCES trimestres (trimestre_id) ON DELETE CASCADE,
    INDEX idx_preguntas_servicio_trimestre (servicio_id, trimestre_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- ------------------------------------------------------------------------
-- 7. RESPUESTAS_ENCUESTA
-- ------------------------------------------------------------------------
CREATE TABLE respuestas_encuesta (
    respuesta_id     INT AUTO_INCREMENT PRIMARY KEY,
    pregunta_id      INT NOT NULL,
    atencion_id      INT NULL,
    valor_respuesta  VARCHAR(500) NOT NULL,
    fecha_respuesta  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_respuestas_pregunta FOREIGN KEY (pregunta_id)
        REFERENCES preguntas_encuesta (pregunta_id) ON DELETE CASCADE,
    CONSTRAINT fk_respuestas_atencion FOREIGN KEY (atencion_id)
        REFERENCES atenciones (atencion_id) ON DELETE SET NULL,
    CONSTRAINT chk_valor_respuesta_no_vacio CHECK (TRIM(valor_respuesta) <> ''),
    INDEX idx_respuestas_pregunta (pregunta_id),
    INDEX idx_respuestas_atencion (atencion_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

SET FOREIGN_KEY_CHECKS = 1;
```

Anexo 2 - Capturas de Pantalla del Formulario de Registro de Atenciones

Formulario del frontend (React + Tailwind CSS) utilizado por el operador de la OGBU para registrar la toma de datos del estudiante en los tres servicios: Alimentacion (Comedor), Servicio Social y Residencias Universitarias. Incluye busqueda rapida por codigo de estudiante (con validacion contra la base de datos real), seleccion del servicio, y campos de texto libre para motivo de atencion y diagnostico/accion tomada.

Estado al cierre de julio de 2026: el sistema se encontraba en etapa de diseño. El desarrollo y las pruebas funcionales que se documentan en este anexo se realizaron en agosto de 2026. El aplicativo aun no ha sido implementado en produccion ni puesto en uso operativo por las unidades de la OGBU.

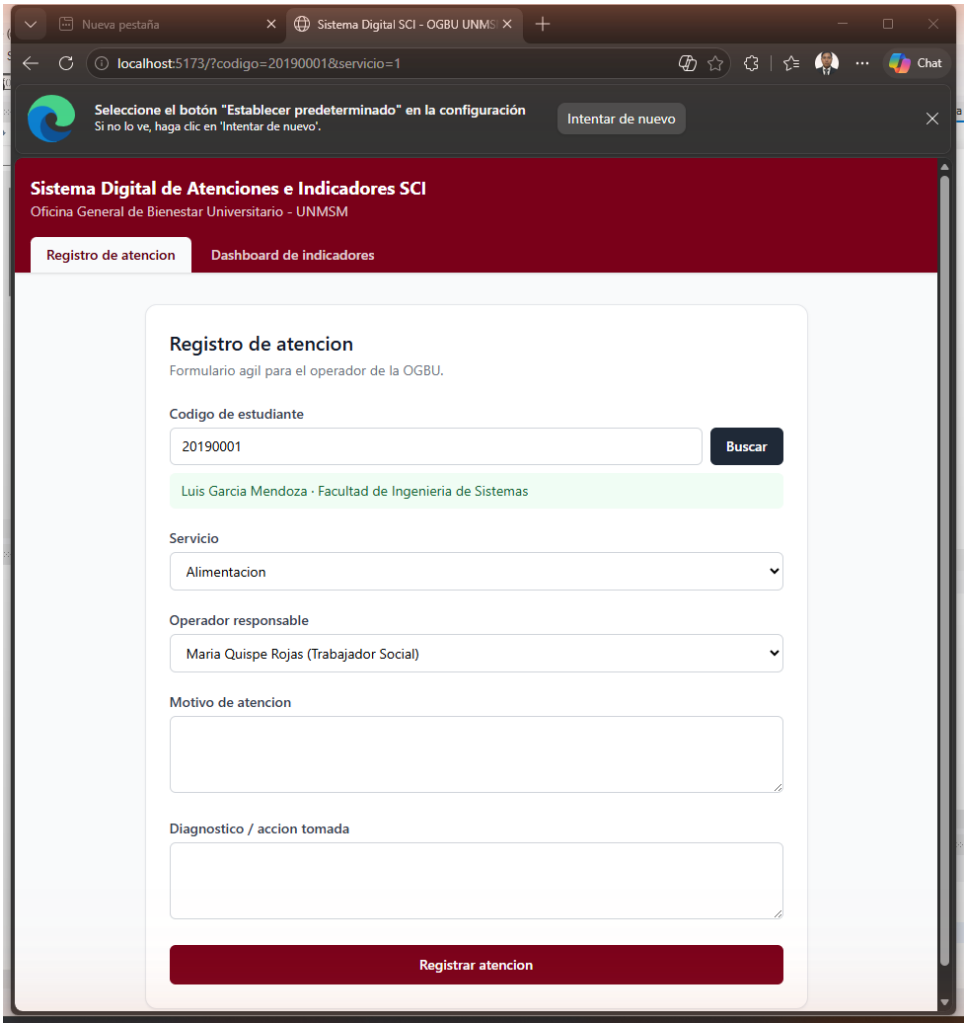


Figura 1. Formulario de Registro de Atencion con estudiante encontrado (codigo 20190001) y servicio Alimentacion seleccionado. Captura tomada sobre la aplicacion en ejecucion (localhost:5173), consumiendo datos reales de la API backend.

Elementos visibles en la evidencia: (1) campo de busqueda por codigo de estudiante con boton "Buscar" que consulta GET /api/estudiantes/:codigo; (2) confirmacion visual en verde del estudiante encontrado (nombre y facultad); (3) desplegable "Servicio" poblado dinamicamente desde GET /api/servicios con las tres opciones: Alimentacion, Residencias Universitarias y Servicio Social; (4) selector de operador responsable; (5) areas de texto para motivo de atencion y diagnostico/accion; (6) boton de guardado que invoca POST /api/atenciones y muestra retroalimentacion visual (mensaje de exito o error).

Anexo 3 - Capturas de Pantalla del Dashboard de Indicadores SCI

Panel de indicadores que calcula en tiempo real, a partir de datos de prueba (dummy) registrados durante las pruebas funcionales del sistema, el Indicador de Cobertura de Atenciones (ICA) y el Indice de Satisfaccion Estudiantil (ISE) requeridos por el Sistema de Control Interno (SCI). Estos datos no corresponden a atenciones reales de estudiantes; ver recuadro de Estado y Alcance en la caratula.

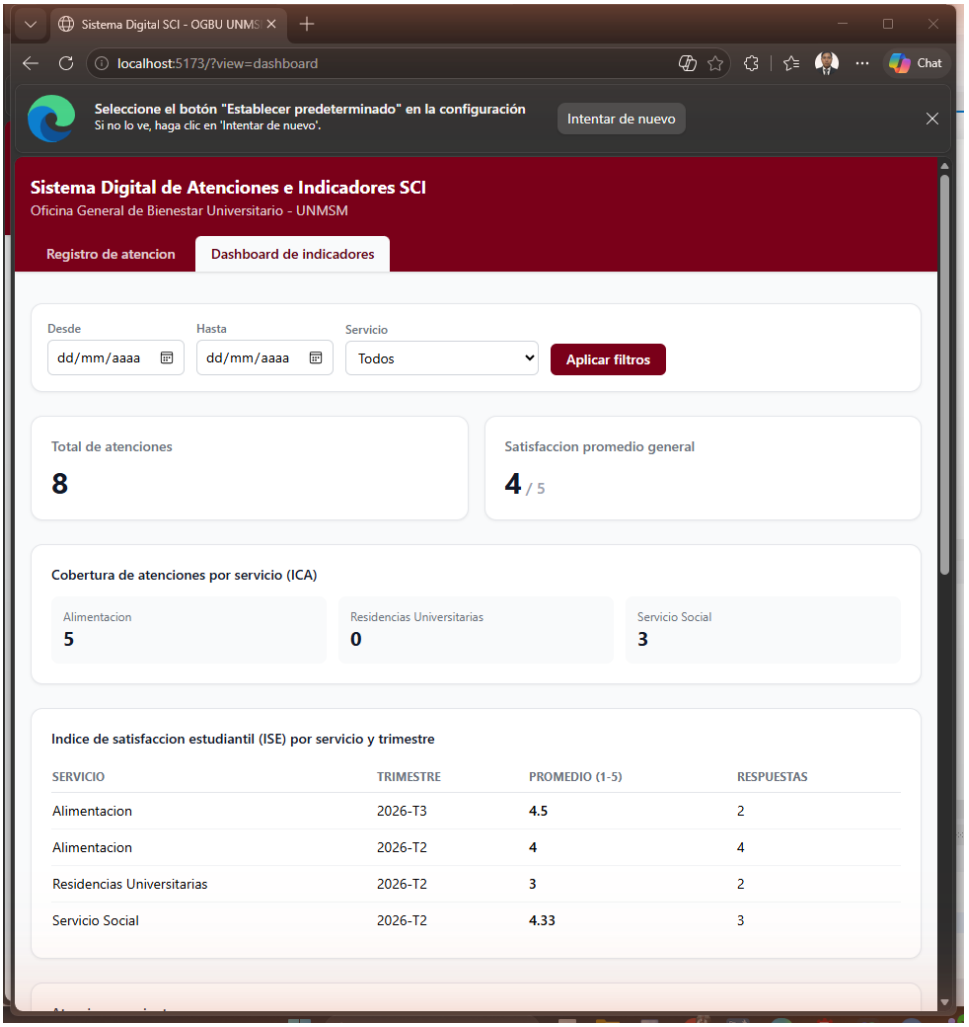


Figura 2. Dashboard de Indicadores SCI mostrando KPIs generales, cobertura de atenciones por servicio (ICA) e indice de satisfaccion estudiantil por servicio y trimestre (ISE), calculados automaticamente por el endpoint GET /api/indicadores/dashboard sobre datos de prueba (dummy). No corresponden a atenciones reales de estudiantes.

3.1 Formulas de calculo

- ICA (Indicador de Cobertura de Atenciones): COUNT(atenciones) agrupado por servicio, dentro de un rango de fechas filtrable (por defecto, ultimos 90 dias).
- ISE (Indice de Satisfaccion Estudiantil): AVG(valor_respuesta) de las preguntas tipo Likert_1_5, agrupado por servicio y trimestre, redondeado a 2 decimales.
- KPI "Satisfaccion promedio general": promedio ponderado de todos los ISE por servicio/trimestre, ponderado por numero de respuestas.

En la captura se observa, por ejemplo, el servicio Alimentacion con 5 atenciones de prueba en el rango de fechas consultado (ICA) y un ISE de 4.5/5 en el trimestre 2026-T3 (2 respuestas) y 4/5 en 2026-T2 (4 respuestas).

3.2 Correspondencia con los indicadores previstos en el PAA-SMC

Comparacion entre los indicadores exigidos por la medida de control (Plan Anual de Auditoria - Seguimiento de Medidas de Control) y los efectivamente desarrollados en el sistema:

Indicador previsto (PAA-SMC)	Indicador desarrollado	Estado
Porcentaje de atencion	ICA - Indicador de Cobertura de Atenciones (conteo de atenciones por servicio, filtrable por rango de fechas)	**Implementado**
Tiempo promedio de atencion por tipo de servicio	No se ha desarrollado un indicador especifico.	**Pendiente**
Niveles de satisfaccion	ISE - Indice de Satisfaccion Estudiantil (promedio de respuestas Likert 1-5 por servicio y trimestre)	**Implementado**

Nota sobre el indicador pendiente: para calcular el tiempo promedio de atencion por servicio se requiere registrar el momento de cierre de cada atencion (adicional al campo estado_atencion ya existente), de modo que el sistema pueda obtener la diferencia entre la fecha de registro (fecha_atencion) y la fecha de cierre, agrupada por servicio. Esta funcionalidad no ha sido implementada y queda pendiente para una siguiente iteracion del sistema.

Anexo 4 - Especificacion de APIs Backend

Endpoints REST desarrollados en Node.js/Express, con validacion de entrada (express-validator), consultas parametrizadas (prevencion de inyeccion SQL) y manejo centralizado de errores. Base URL local: http://localhost:4000/api

4.1 POST /api/atenciones

Metodo y ruta	`POST /api/atenciones`
Descripcion	Registra una nueva atencion. Resuelve el estudiante por codigo y el servicio por ID, asigna la fecha/hora automaticamente en el servidor, y evalua si corresponde disparar la encuesta de satisfaccion del trimestre vigente.
Parametros / body	• `codigo_estudiante (string, requerido, alfanumerico, max 20)` • `servicio_id (entero, requerido)` • `descripcion_motivo (string, requerido, max 2000)` • `diagnostico_accion (string, opcional, max 2000)` • `usuario_responsable_id (entero, requerido)`
Validaciones	• Codigo de estudiante debe existir y estar activo (404/409 si no) • Servicio debe existir y tener estado Activo (404/409 si no) • Usuario responsable debe existir y estar activo (404 si no) • descripcion_motivo no puede estar vacio ni exceder 2000 caracteres
Respuesta exitosa	`201 Created` `{ ok: true, atencion_id, fecha_atencion, estado_atencion, trigger_encuesta: true|false, trimestre_id, preguntas_encuesta: [...] }`
Errores posibles	• 400 Bad Request - datos invalidos (detalle por campo) • 404 Not Found - estudiante, servicio o usuario no existen • 409 Conflict - estudiante/servicio inactivo

4.2 GET /api/atenciones

Metodo y ruta	`GET /api/atenciones`
Descripcion	Lista atenciones con filtros opcionales, usada por la tabla de "Atenciones recientes" del dashboard.
Parametros / body	• `fecha_inicio (fecha ISO 8601, opcional)` • `fecha_fin (fecha ISO 8601, opcional)` • `servicio_id (entero, opcional)` • `estado_atencion (En Proceso | Completado, opcional)` • `limite (entero 1-200, opcional, default 50)` • `pagina (entero >=1, opcional, default 1)`
Validaciones	• Todos los filtros son validados por tipo/formato antes de construir la consulta parametrizada
Respuesta exitosa	`200 OK`

	`{ ok: true, total_retornado, atenciones: [...] }`
Errores posibles	• 400 Bad Request - filtros con formato invalido

4.3 GET /api/indicadores/dashboard

Metodo y ruta	**`GET /api/indicadores/dashboard`**
Descripcion	Calcula y retorna los indicadores de gestion del SCI: ICA e ISE, mas KPIs generales.
Parametros / body	• `fecha_inicio (fecha ISO 8601, opcional, default: hoy - 90 dias)` • `fecha_fin (fecha ISO 8601, opcional, default: hoy)` • `servicio_id (entero, opcional - filtra ambos indicadores a un solo servicio)`
Validaciones	• Fechas deben tener formato ISO 8601 valido

Respuesta exitosa	`200 OK`
	`{ ok: true, rango: {...},`
	`kpis: { total_atenciones, promedio_satisfaccion_general },`
	`indicador_cobertura_atenciones: [...],`
	`indice_satisfaccion_estudiantil: [...] }`

Errores posibles	• 400 Bad Request - fechas o servicio_id invalidos

4.4 POST /api/encuestas/responder

Metodo y ruta	**`POST /api/encuestas/responder`**
Descripcion	Guarda de forma masiva (transaccional) las respuestas de una encuesta de satisfaccion. Si una respuesta es invalida, no se guarda ninguna del lote.
Parametros / body	• `respuestas (array, 1-50 elementos, requerido), cada elemento:` • `pregunta_id (entero, requerido)` • `atencion_id (entero, opcional/nullable)` • `valor_respuesta (string, requerido, max 500)`
Validaciones	• Cada pregunta_id debe existir y estar activa • Si tipo_respuesta = Likert_1_5, el valor debe ser un entero entre 1 y 5 • Operacion ejecutada dentro de una transaccion (BEGIN/COMMIT/ROLLBACK)
Respuesta exitosa	`201 Created` `{ ok: true, respuestas_guardadas, respuesta_ids: [...] }`
Errores posibles	• 400 Bad Request - respuesta Likert fuera de rango 1-5 • 404 Not Found - pregunta_id no existe • 409 Conflict - pregunta ya no esta activa

4.5 GET /api/estudiantes/:codigo

Metodo y ruta	**`GET /api/estudiantes/:codigo`**

Descripcion	Busqueda rapida de un estudiante por su codigo, usada para autocompletar el formulario de registro.
Parametros / body	• `codigo (parametro de ruta, alfanumerico, requerido)`
Validaciones	• Formato de codigo validado (alfanumerico, max 20 caracteres)
Respuesta exitosa	`200 OK` `{ ok: true, estudiante: { estudiante_id, codigo_estudiante, nombres, apellidos, facultad, correo_institucional, estado } }`
Errores posibles	• 404 Not Found - codigo no existe • 409 Conflict - estudiante inactivo

4.6 GET /api/servicios

Metodo y ruta	**`GET /api/servicios`**
Descripcion	Lista los servicios OGBU activos (Alimentacion, Servicio Social, Residencias Universitarias), usada para poblar el desplegable del formulario.
Parametros / body	• `Sin parametros`
Validaciones	• No aplica
Respuesta exitosa	`200 OK` `{ ok: true, servicios: [{ servicio_id, nombre_servicio, descripcion }, ...] }`
Errores posibles	• 500 Internal Server Error (fallo de conexion a BD)

4.7 GET /api/health

Metodo y ruta	**`GET /api/health`**
Descripcion	Verificacion de disponibilidad del servicio backend (health check).
Parametros / body	• `Sin parametros`
Validaciones	• No aplica
Respuesta exitosa	`200 OK` `{ ok: true, servicio: "sci-ogbu-backend" }`
Errores posibles	• -

4.8 Seguridad transversal

- Consultas 100% parametrizadas (placeholders), sin concatenacion de entrada de usuario - previene inyeccion SQL.
- Validacion y sanitizacion de entrada con express-validator en cada ruta (tipos, longitudes, formatos, whitelists).

- Cabeceras HTTP endurecidas (helmet) y CORS restringido a origenes configurados explicitamente.
- Rate limiting general (300 req/15min) y reforzado en endpoints de escritura (100 req/15min).
- Manejo centralizado de errores que no filtra detalles internos (stack traces, SQL) al cliente.